# Roteiro de retomada — Catálogo do karaokê

**Versão 2 · 27/10/2026** (substitui a versão de entrega de 20/10/2026; tarefas e aceites não mudaram).
**Programação III · Ensino Médio Integrado — Técnico em Informática · IFSul — Campus Passo Fundo**
**Professor:** Élder F. F. Bernardi · **Modalidade:** duplas **Tutor da atividade:** Prof. Leonardo Constantin

**Organização:** 14 duplas por turma, 150min em aula e 1h extraclasse por encontro. Quatro encontros: 600 + até 240min = 14h de desenvolvimento; apresentação em outro encontro de 150min. Setup essencial acompanhado em aula.

## Sumário


1. Para que serve esta base
2. 1. Preparar e executar
3. 2. Encontrar cada responsabilidade
4. 3. Conferir o contrato antes de programar
5. 4. Integrar cadastro — R2 e R5
6. 5. Integrar edição — R3 e R5
7. 6. Integrar exclusão — R4 e R5
8. 7. Conferir o núcleo e publicar — R1, R6, R7 e R8
9. 8. Consolidar entre encontros — proposta de até 60min


## Para que serve esta base

A base de retomada disponibilizada pelo professor no Classroom é opcional e não reduz a nota. Ela permite retomar a integração sem reconstruir toda a infraestrutura. Leitura, listagem, player, API e DAOs já vêm preparados, junto com um `AGENTS.md` que orienta o agente de IA a agir como tutor. As tarefas da dupla são integrar **cadastro, edição e exclusão na interface**, tratar respostas e atualizar a lista. O núcleo e a compreensão exigidos são os mesmos para quem usa projeto próprio.

↑ Voltar ao sumário

## 1. Preparar e executar

1. Abram a pasta `base_retomada` no editor e confiram o README.
2. Usem **Node.js 22 ou superior**, compatível com as dependências fixadas na base.
3. Executem `npm ci` para instalar as dependências declaradas.
4. A partir de `.env.example`, configurem no servidor `SUPABASE_URL` e `SUPABASE_SECRET_KEY`, com orientação do professor.
5. Preparem um banco de demonstração separado, nesta ordem. Primeiro, as tabelas: se as da atividade anterior já existem, não as recriem; rodem `sql/diagnostico.sql` no SQL Editor e, se alguma linha vier `ajustar`, rodem `sql/ajustes.sql` (não apaga dados) e depois o diagnóstico de novo. Sigam quando não houver nenhuma linha `ajustar` ou `criar`; linhas `conferir` de coluna que aceita vazio não impedem. Se as tabelas ainda não existem, apliquem `sql/schema.sql`. Só depois incluam a amostra com `sql/seed.sql`, conforme o README. O seed desta base é aditivo, não apaga o catálogo; não o substituam por scripts antigos de restauração que removem dados. O passo a passo está na etapa 4 do "Guia passo a passo do Supabase", disponível no Classroom da disciplina.
6. Com ambiente e banco preparados, executem `npm start`. Abram a URL indicada pelo servidor e confirmem lista, seleção e reprodução da música de exemplo.

Os valores reais das variáveis são secretos: não enviar ao Git, ao navegador, a capturas ou a mensagens. A interface usa a API na mesma origem, como `/api/musicas`; não precisa de CORS nem de chave de banco no JavaScript público. Se a leitura falhar, registrem a mensagem e o ponto do fluxo antes de continuar.

↑ Voltar ao sumário

## 2. Encontrar cada responsabilidade

| Arquivo | O que localizar durante a leitura |
|---|---|
| `app.js` | Rotas HTTP, leitura do corpo JSON, chamada de DAO e resposta. |
| `supabaseClient.js` | Configuração do cliente no servidor, sem copiar os valores secretos. |
| `karaoke/artista.js`, `musica.js`, `parte.js` | Objetos usados para representar os dados. |
| `karaoke/DAO/ArtistaDAO.js`, `MusicaDAO.js`, `ParteDAO.js` | Consultas, alterações e transformação de linhas do banco em objetos. |
| `sql/schema.sql`, `sql/seed.sql` | Relações entre tabelas e repertório inicial de demonstração. |
| `sql/diagnostico.sql`, `sql/ajustes.sql` | Conferir e corrigir, sem apagar dados, tabelas que já existiam. |
| `AGENTS.md` | Orientações para o agente de IA agir como tutor: explica e pergunta, sem entregar a solução. |
| `public/index.html`, `estilo.css` | Campos, lista, mensagens e botões da interface. |
| `public/catalogo.js` | Leitura, atualização da lista, seleção e player já preparados. |
| `public/admin.js` | Pontos de integração a completar: cadastro, edição, exclusão e feedback. |

Uma pessoa acompanha o clique até a rota; a outra acompanha a rota até o DAO e o banco. Depois troquem e expliquem o caminho de volta à tela.

### Se vocês vieram do repositório da aula

A base tem os mesmos arquivos do repositório da aula, mas alguns ficam em outras pastas. O código dos DAOs é o mesmo; só mudam os caminhos dos `require`.

| Arquivo | Repositório da aula | Base de retomada |
|---|---|---|
| `supabaseClient.js`, `app.js` | raiz | raiz |
| `Artista`, `Musica`, `Parte` | `karaoke/artista.js`, `karaoke/musica.js`, `karaoke/parte.js` | `karaoke/artista.js`, `karaoke/musica.js`, `karaoke/parte.js` |
| `ArtistaDAO`, `ParteDAO`, `MusicaDAO` | `DAO/` | `karaoke/DAO/` |
| testes | `testes/` | `testes/` (a base já traz os seus) |
| SQL | `sql/` | `sql/` |

Exemplo: o `MusicaDAO.js` importa o cliente com `require('../supabaseClient')` no repositório da aula e com `require('../../supabaseClient')` na base. O "Guia passo a passo do Supabase", disponível no Classroom da disciplina, traz no Anexo a tabela completa de `require` e como calcular o caminho em projeto próprio.

↑ Voltar ao sumário

## 3. Conferir o contrato antes de programar

| Operação | Contrato da base |
|---|---|
| Listar | `GET /api/musicas`: itens com `id`, `nome`, `artista` e `totalPartes`. |
| Consultar | `GET /api/musicas/:id`: música e suas partes; cada parte tem `letra`, `tempoEspera`, `tag` e `ordem`. |
| Cadastrar | `POST /api/musicas`: JSON `{ "nome": "Música de teste", "artista": "Artista de teste" }`. |
| Editar | `PUT /api/musicas/:id`: os mesmos campos de texto do cadastro. |
| Excluir | `DELETE /api/musicas/:id`: ID da música escolhida no caminho da URL. |

**Atenção:** no envio, `artista` é uma **string**; na resposta de música, `artista` é um **objeto** com `id` e `nome`. Para preencher um campo de texto, localizem o nome dentro do objeto. Não enviem o objeto inteiro no POST/PUT. Leiam a rota e a resposta real para conferir status e mensagem de erro.

↑ Voltar ao sumário

## 4. Integrar cadastro — R2 e R5

1. Em `admin.js`, localizem a tarefa do formulário e os campos existentes no HTML.
2. Leiam nome/artista, verifiquem os obrigatórios e impeçam recarregamento involuntário do formulário.
3. Planejem um POST com corpo JSON e o cabeçalho correspondente; aguardem a resposta.
4. Separem resposta de sucesso de erro HTTP/falha de conexão. Só anunciem cadastro após confirmação.
5. Reutilizem a atualização da lista já fornecida; anotem ID e resultado de uma nova consulta.

> **Pergunte ao agente.** *"Explique o que acontece se eu esquecer o evento.preventDefault() no submit do formulário. Antes de responder, peça que eu preveja."* Depois: *"Qual a diferença entre a resposta ter response.ok falso e o fetch lançar uma exceção? Dê um exemplo de cada situação no meu projeto. Use como fonte a seção 4 do roteiro_retomada.md e os Padrões 3 e 4 do guia_interface_crud.md, na página do trabalho: https://elderbernardiifsul.github.io/programacao-iii/integrador-2026/."*

**Aceite:** registro aparece após recarregar a página e permanece após reiniciar o servidor. Nome ou artista vazio não gera cadastro válido. Troquem a condução.

↑ Voltar ao sumário

## 5. Integrar edição — R3 e R5

1. Localizem como guardar o ID selecionado e preencher nome e `artista.nome`.
2. Diferenciem o estado de edição do cadastro e permitam cancelar a edição.
3. Enviem PUT ao ID selecionado, confiram a resposta e atualizem a lista.

> **Pergunte ao agente.** *"Estou no modo edição e, ao salvar, é criada uma música nova em vez de alterar a existente. Que perguntas devo me fazer para achar o erro? Use como fonte a seção 5 do roteiro_retomada.md e o Padrão 5 do guia_interface_crud.md, na página do trabalho: https://elderbernardiifsul.github.io/programacao-iii/integrador-2026/."*

**Aceite:** o mesmo ID passa a ter o novo nome/artista após nova consulta; editar não cria outra música. Expliquem onde ID e campos entram na requisição.

↑ Voltar ao sumário

## 6. Integrar exclusão — R4 e R5

1. Identifiquem o registro e peçam confirmação simples antes do DELETE.
2. Aguardem resposta, mostrem erro quando houver e atualizem lista/seleção após sucesso.
3. Testem cancelamento e exclusão usando uma música criada para teste.

> **Pergunte ao agente.** *"Revise a minha função de exclusão (colada abaixo). Aponte a linha do problema e explique, sem reescrever a função. Use como fonte a seção 6 do roteiro_retomada.md e o Padrão 6 do guia_interface_crud.md, na página do trabalho: https://elderbernardiifsul.github.io/programacao-iii/integrador-2026/."*

**Aceite:** cancelar preserva o dado; confirmar remove o registro em nova consulta. Preservem a música do seed usada para demonstrar o player.

↑ Voltar ao sumário

## 7. Conferir o núcleo e publicar — R1, R6, R7 e R8

- Cadastrem música sem partes: ela é válida, pode ser editada/excluída, mostra a situação e mantém Tocar indisponível.
- Reproduzam uma música com partes fornecidas; confiram vínculo e ordem, sem adicionar letras como requisito.
- Iniciem a configuração da publicação no primeiro encontro e retomem o CRUD na URL pública conforme os marcos.
- Registrem ID antes/depois de reiniciar o serviço; expliquem por que uma lista local não comprova persistência.
- Completem README com execução, URL, variáveis sem valores e uma decisão; registrem um teste por marco.

Para a interface com `fetch`, consultem o "Guia de interface — CRUD na tela com fetch", disponível no Classroom da disciplina. Para a publicação, o "Guia de deploy na Render", também disponível no Classroom da disciplina, traz o passo a passo.

Cada pessoa acompanha uma operação e realiza ou prevê uma pequena alteração equivalente com consulta. Podem explicar oralmente, apontando o código ou por anotação/desenho acompanhado de demonstração. Se travar, registrem o primeiro ponto que falhou e peçam apoio em aula. O trabalho essencial tem espaço em aula.

↑ Voltar ao sumário

## 8. Consolidar entre encontros — proposta de até 60min

| Após o marco | Tarefa delimitada de revisão/teste/documentação |
|---|---|
| M1 | Rever mapa do fluxo (20min), completar execução no README (20min), anotar dúvida/próximo passo (20min). |
| M2 | Repetir cadastro/reconsulta (20min), registrar teste (20min), revisar trecho para explicar (20min). |
| M3 | Repetir checklist conhecido (30min), registrar primeira falha (15min), preparar explicação (15min). |
| M4 | Revisar README (20min), ensaiar percurso (20min), organizar evidências (20min). |

Se houver impedimento de acesso, registrem para organizar apoio com o professor; não ampliem a tarefa além da hora prevista. Não é necessário adicionar extras.

↑ Voltar ao sumário